Python Exam Cheatsheet

Front • Labs 6–7 • Functions, dictionaries, files, input and tests

A reliable way to start

• Mark the input, required output, special cases, and exact return type (dict, list, int or str).

• If an assert checks a returned value, use return. Use print only when the prompt asks for displayed output.

• Build the result in four steps: initialize → loop → update → return. Trace one small example by hand.

Dictionary: choose key and value

```
# course_id -> (course_name, credits)
courses = {'2301117': ('Calculus I', '4')}
info = courses.get(course_id)
if info is None:
return 'Unknown'
return info[0] + ' ' + info[1]
```

• Read with d[key] or d.get(key); test membership with key in d. Keep IDs as strings when leading zeroes or slicing matter.

Read a file and count words

```
counts = {}
with open(file_name) as f:
for line in f:
for word in line.split():
word = word.strip(':?,.')
counts[word] = counts.get(word, 0) + 1
return counts
```

• split() separates on whitespace. strip(chars) removes those characters only from the beginning and end of each token.

Read name-score pairs and rank

```
scores = {}
with open(file_name) as f:
for line in f:
name, score = line.split()
scores[name] = int(score)
return sorted(scores, key=scores.get, reverse=True)
```

• Convert numeric text with int(). sorted(dict, key=dict.get, reverse=True) returns keys ordered by their values.

Top three, filters and totals

```
top3 = sorted(scores, key=scores.get, reverse=True)[:3]
popular = [name for name, n in sports.items() if n >=
15]
total = sum(sports.values())
sports[sport] = sports.get(sport, 0) + players
```

• Use accumulation when a name can appear on multiple file lines. The last line adds this row to the existing total.

Worked trace: contest scores

```
File row: S01 8 9 7 60
Total = 8 + 9 + 7 + 60 = 84
scores['S01'] = 84
Repeat for each row, then sort keys by score
Top 3 = sorted(scores, key=scores.get,
reverse=True)[:3]
```

• Separate parsing from ranking: get_dict builds the score dictionary; find_top3 only sorts that dictionary.

Data patterns & checks

Front • Nested dictionaries, input validation and exact checks

Nested dictionaries: stock and menu

```
# stock[product_id][size] = quantity
if product_id not in stock:
stock[product_id] = {}
stock[product_id][size] = stock[product_id].get(size, 0)
+ qty

# menu[drink][size] = price
total = menu[drink][size] * number
```

• Read nested data from the outside in: first key → inner dictionary → second key → value. Match the required types (int ID, string size, etc.).

Mini trace: count word frequency

```
Line: 'Hi, Ana. Hi'
split() -> ['Hi,', 'Ana.', 'Hi']
strip(':?,.') -> ['Hi', 'Ana', 'Hi']
count 'Hi' twice; count 'Ana' once
Result: {'Hi': 2, 'Ana': 1}
```

• At each token, ask: is it already a key? Add 1 if yes; otherwise start it at 1.

Input, then validate

```
drink, size, number = input('...').split()
number = int(number)
if drink not in menu:
print('Drink not available.')
elif size not in menu[drink]:
print('Incorrect size.')
else:
print(f'Total : {menu[drink][size] * number}')
```

• Check the drink before looking up its size. Use the price table from the prompt; do not guess values.

Trace one menu lookup

```
Input: Latte L 2
menu['Latte']['L'] -> 75
number -> int('2') -> 2
total = 75 * 2 -> 150
Output: Total : 150
```

• A lowercase name can be a different key. Match the prompt's capitalization and check invalid drink/size before indexing.

Student ID → tuition lookup

```
faculty = student_id[-2:] # last 2 digits
level_digit = student_id[2] # third digit
year = int(student_id[:2])
fee = table[group][level][term][id_group]
```

• Third digit: 3/4 = Bachelor, 7 = Graduate. Terms 1/2 map to S1_2; term 3 maps to S3.

• Year prefixes: 48–49 = ID_1, 50–55 = ID_2, 56+ = ID_3. Faculty group 1: 21, 23, 25, 28, 30–39, 53. Group 2: 22, 24, 26, 27, 29, 34, 40, 51.

Before submitting

• Check capitalization, punctuation, spaces, and trailing newlines: string assertions compare every character.

• Handle missing keys, empty lists, and zero counts. Do not delete dictionary keys while iterating over that dictionary.

OOP: Think in Objects

Back • Lab 8 • Classes, attributes, methods and object collections

Start from real things

• Nouns that represent things become classes. Their data becomes attributes; their actions become methods.

• A class is a blueprint; an object is one instance. self means “this object,” the instance using the method.

```
class Rectangle:
def __init__(self, length, width):
self.length = length
self.width = width
def area(self):
return self.length * self.width
def perimeter(self):
return 2 * (self.length + self.width)

r = Rectangle(5, 3)
r.area() # self is r
```

__init__ and how values move

• Rectangle(5, 3) creates an object and calls __init__ automatically. Python passes that object as self; 5 and 3 become length and width.

• Calling obj.method(x) is conceptually Class.method(obj, x). Do not pass self yourself.

Trace self in a call

```
r = Rectangle(5, 3)
r.area()
# same idea as Rectangle.area(r)
# inside area: self is r
# self.length is 5; self.width is 3
# result returned: 15
```

One object can contain another

```
class Book:
def __init__(self, title, author):
self.title = title
self.author = author # an Author object

book.author.name # follow the reference to the name
```

• Use dot access for object attributes. If book.author stores an Author object, read its name as book.author.name, not book.author[0].

Collection of objects

• A list may hold Member objects: loop with for member in self.members, then read member.name or member.role.

• A dictionary may map an ID to an object: self.books[book.isbn] = book. Later, self.books[isbn] is the Book object.

Bank and Team patterns

```
# name -> Account object
self.accounts[name] = Account(balance)
self.accounts[src].withdraw(amount)
self.accounts[dst].deposit(amount)

# list of Member objects
for member in self.members:
if member.name == name:
self.members.remove(member)
break
```

• For role counts, use counts[member.role] = counts.get(member.role, 0) + 1. For a percentage, count / total * 100, then round(..., 2).

Worked example: Library

Trace the data flow from Author to Book to Library, then check what each test expects.

1. Decide what each object stores

```
Author: name, birth_year
Book: title, isbn, author (Author object),
available_copies
Library: books (dict: ISBN -> Book object)
```

• The relationships are has-a: a Book stores an Author object; a Library stores Book objects.

2. Create objects and connect them

```
author = Author('John Doe', 1980)
book = Book('Intro to Python', '978-...', author, 10)
library = Library()
library.add_book(book)
```

• book.author refers to the same Author object. The library dictionary key is the ISBN; its value is the whole Book.

3. Add a new book or merge a duplicate

```
def add_book(self, book):
if book.isbn in self.books:
self.books[book.isbn].available_copies +=
book.available_copies
else:
self.books[book.isbn] = book
```

• For a repeated ISBN, update the existing Book object; replacing it would lose the earlier copy count.

4. Follow references to search and total

```
total = sum(b.available_copies for b in
self.books.values())
by_author = [b.title for b in self.books.values()
if b.author.name == author_name]
match = [b for b in self.books.values()
if b.title == title]
```

• Search path: Library → Book → Author → name. Title is directly on Book.

5. Trace one test

```
book starts with 10 copies
borrow_copy(): 10 -> 9
return_copy(): 9 -> 10
add duplicate ISBN with 2 copies: 10 -> 12
```

• A method that “returns” a message must return a string, not print it. Update the count first, then return the exact expected text.

Exact strings matter

```
def __str__(self):
return f'Name: {self.name}, Role: {self.role}'

# print(obj) calls __str__ automatically
# '\n' inserts a newline in a string
```

• When an assert fails, inspect the first failing assert. For strings, compare line breaks, spaces, punctuation and the final newline.

Exam workflow: identify types → write __init__ → implement one method at a time → trace a test → compare the returned value.